
大数据与人工智能
自动化学院

Chapter03.9 模型的保存加载、模型微调、GPU使用

3.9-1 导言

- 通过前面的9节，我们就可以通过Pytorch搭建一个模型并且进行有效的训练，
- 而模型搭建完了之后我们要保存下来，以备后面的使用，并且在大型任务中我们不可能从头自己搭建模型，往往需要模型的迁移，
- 为了提高训练效率，往往需要使用GPU

大纲如下：

- 模型的保存与加载
- 模型的finetune
- GPU使用

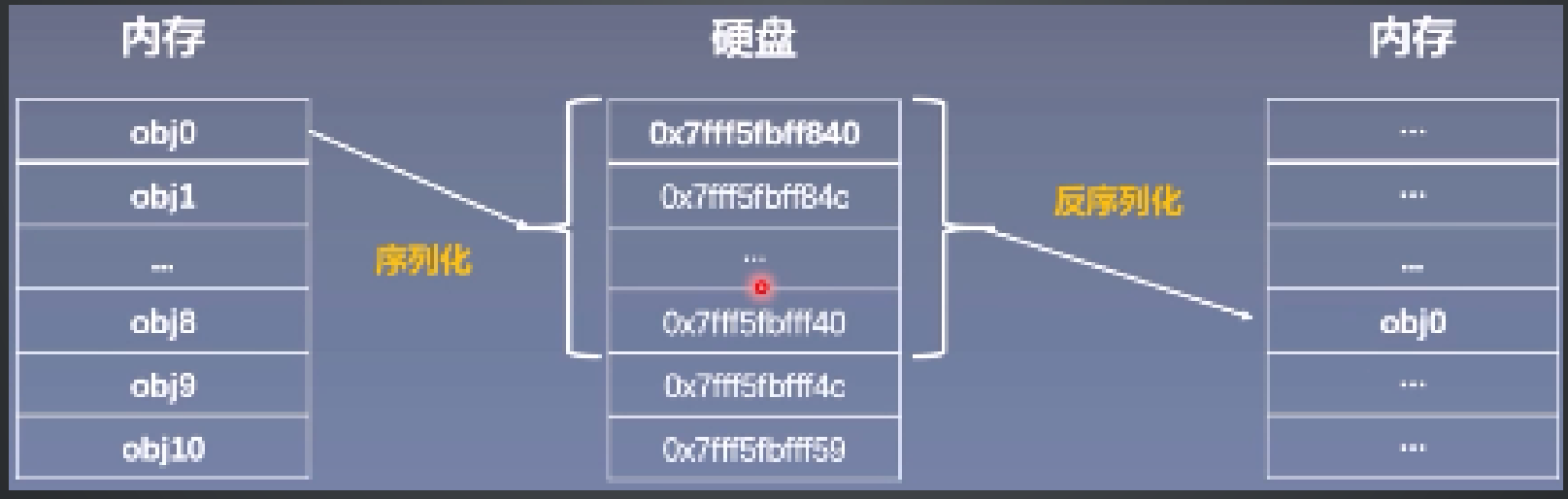
3.9-2. 模型的保存与加载

我们的建立的模型训练好了是需要保存的，以备我们后面的使用

- 如何保存模型和加载模型呢？ 主要分为三块：
  - 首先介绍一下序列化和反序列化
  - 然后介绍模型保存和加载的两种方式
  - 最后是断点的续训练技术

3.9-2.1 序列化与反序列化

- 序列化就是内存中的某一个对象保存到硬盘当中，以二进制序列的形式存储下来，这就是一个序列化的过程。
- 反序列化，就是将硬盘中存储的二进制的数，反序列化到内存当中，得到一个相应的对象，这样就可以再次使用这个模型了。



序列化和反序列化的目的就是将我们的模型长久的保存。

Pytorch中序列化和反序列化的方法：

- `torch.save(obj, f)`:obj表示对象，也就是我们保存的数据，可以是模型，张量，dict等等，f表示输出的路径
- `torch.load(f, map_location)`:f表示文件的路径，`map_location`指定存放位置，CPU或者GPU，这个参数挺重要，在使用GPU训练的时候再具体说。

3.9-2.2 模型保存与加载的两种方式

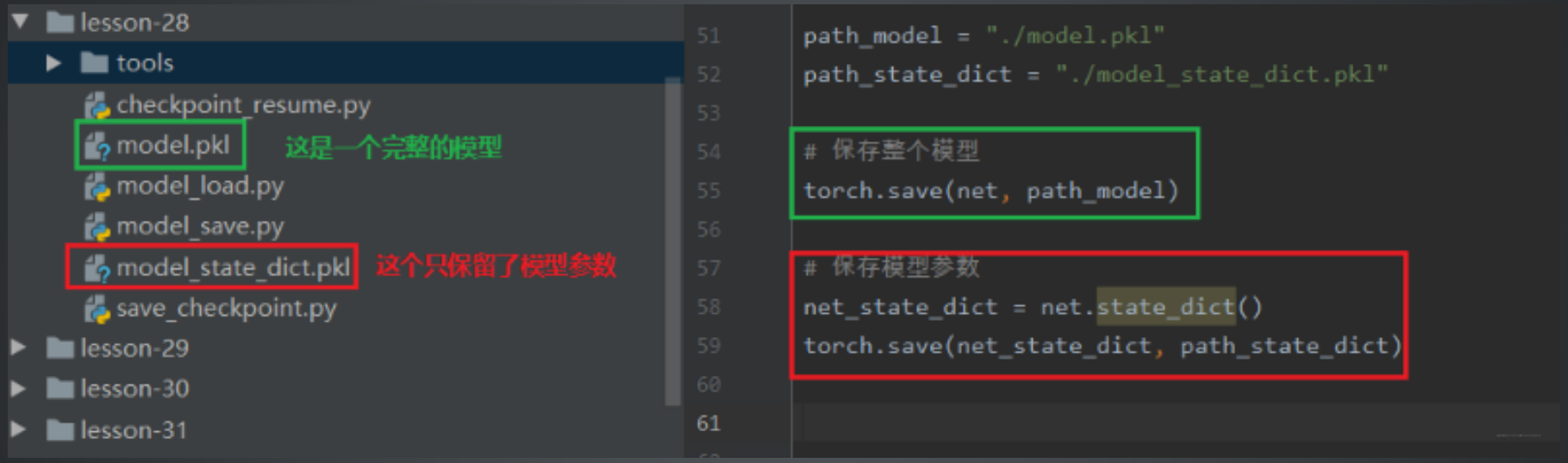
Pytorch的模型保存有两种方法， 一种是保存整个Module，另外一种是保存模型的参数。

- 保存和加载整个Module： `torch.save(net, path)`, `torch.load(fpath)`
- 保存模型参数： `torch.save(net.state_dict(), path)`, `net.load_state_dict(torch.load(path))`
- 第一种方法比较傻瓜，保存整个的模型架构， 比较费时占内存
- 第二种方法是只保留模型上的可学习参数， 等建立一个新的网络结构，然后放上这些参数即可，所以推荐使用第二种。

下面通过代码看看具体怎么使用：
这里先建立一个网络模型：

```
class LeNet2(nn.Module):
    def __init__(self, classes):
        super(LeNet2, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 6, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(6, 16, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
```

下面就是保存整个模型和保存模型参数的方法：

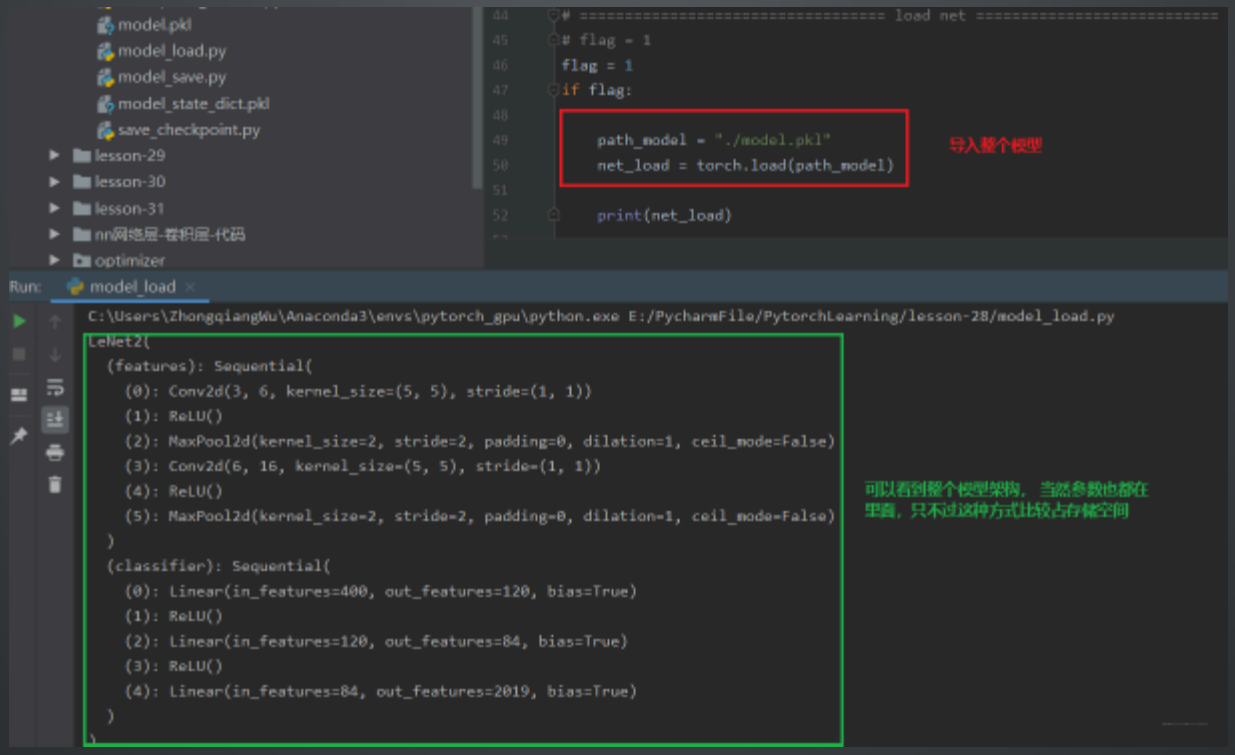


通过上面，我们已经把模型保存到硬盘里面了，那么如果要用的时候，应该怎么导入呢？

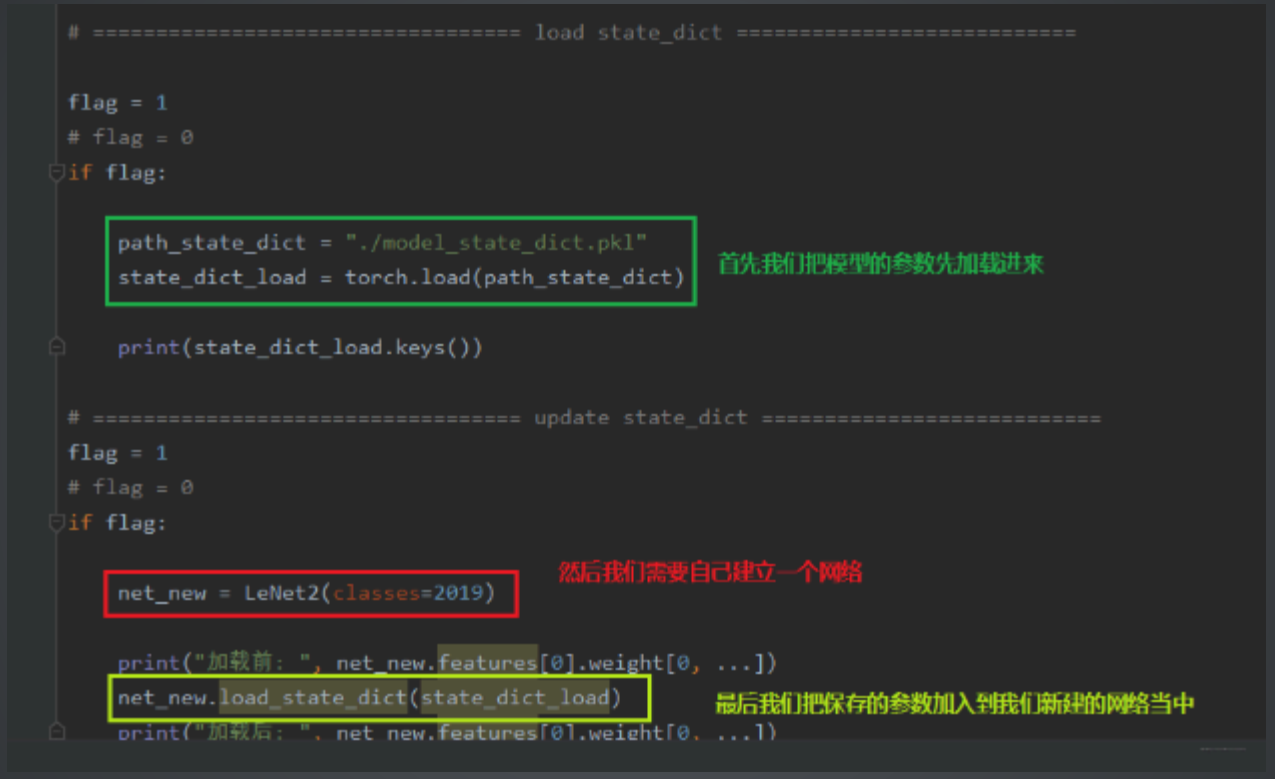
如果我们保存的是整个模型的话， 那么导入的时候就非常简单， 只需要：

```
path_model = "./model.pkl"
net_load = torch.load(path_model)
```

并且我们可以直接打印出整个模型的结构：



下面看看只保留模型参数的话应该怎么再次使用：




```
# ================================== load state_dict ==========================

flag = 1
# flag = 0
if flag:

    path_state_dict = "./model_state_dict.pkl"
    state_dict_load = torch.load(path_state_dict)

    print(state_dict_load.keys())

# ================================== update state_dict ==========================
flag = 1
# flag = 0
if flag:

    net_new = LeNet2(classes=2019)

    print("加载前: ", net_new.features[0].weight[0, ...])
    net_new.load_state_dict(state_dict_load)
    print("加载后: ", net_new.features[0].weight[0, ...])
```

上面就是两种模型加载与保存的方式了，使用起来也是非常简单的，推荐使用第二种。

提问：关于模型的保存与加载，哪种方法更推荐使用？

- A) 保存整个Module
- B) 保存模型参数
- C) 保存数据集
- D) 保存损失函数

参考答案：B

选项解析：

- A) 保存整个Module会保存模型架构，虽然简单但占用内存较多。
- B) 保存模型参数只保留可学习参数，适合在新网络上加载，推荐使用。
- C) 保存数据集与模型保存无关。
- D) 保存损失函数不是常见的保存方式。

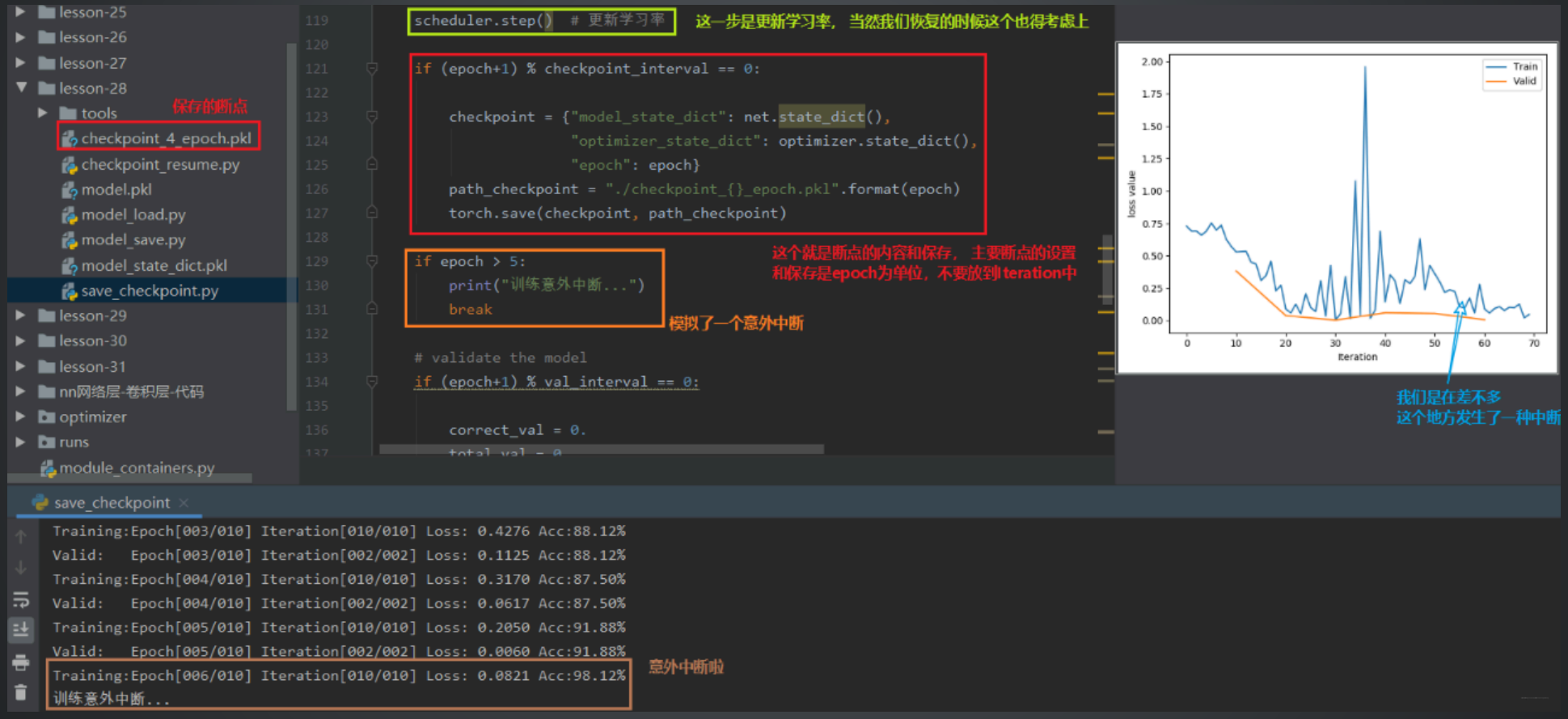
3.9-2.3 模型断点续训练

- 当我们的模型训练的时间非常长，而训练到了中途出现了一些意外情况，比如断电了，当再次来电的时候，我们希望模型在中途的那个地方继续往下训练
- 这就需要我们在模型的训练过程中保存一些断点checkpoint，这样发生意外之后，我们的模型可以从断点处继续训练而不是从头开始。**所以模型训练过程中设置checkpoint也是非常重要的。**

- 那么checkpoint里面需要保留哪些参数呢?
- 再次回忆模型训练的五个步骤： 数据 -> 模型 -> 损失函数 -> 优化器 -> 迭代训练。
- 在这五个步骤中，我们知道样本数据，损失函数这些没法变
- 而在迭代训练过程中，我们模型里面的可学习参数， 优化器里的一些缓存是会变的，
- 所以checkpoint里面需要保存模型的数据，优化器的数据，还有迭代到了第几次。

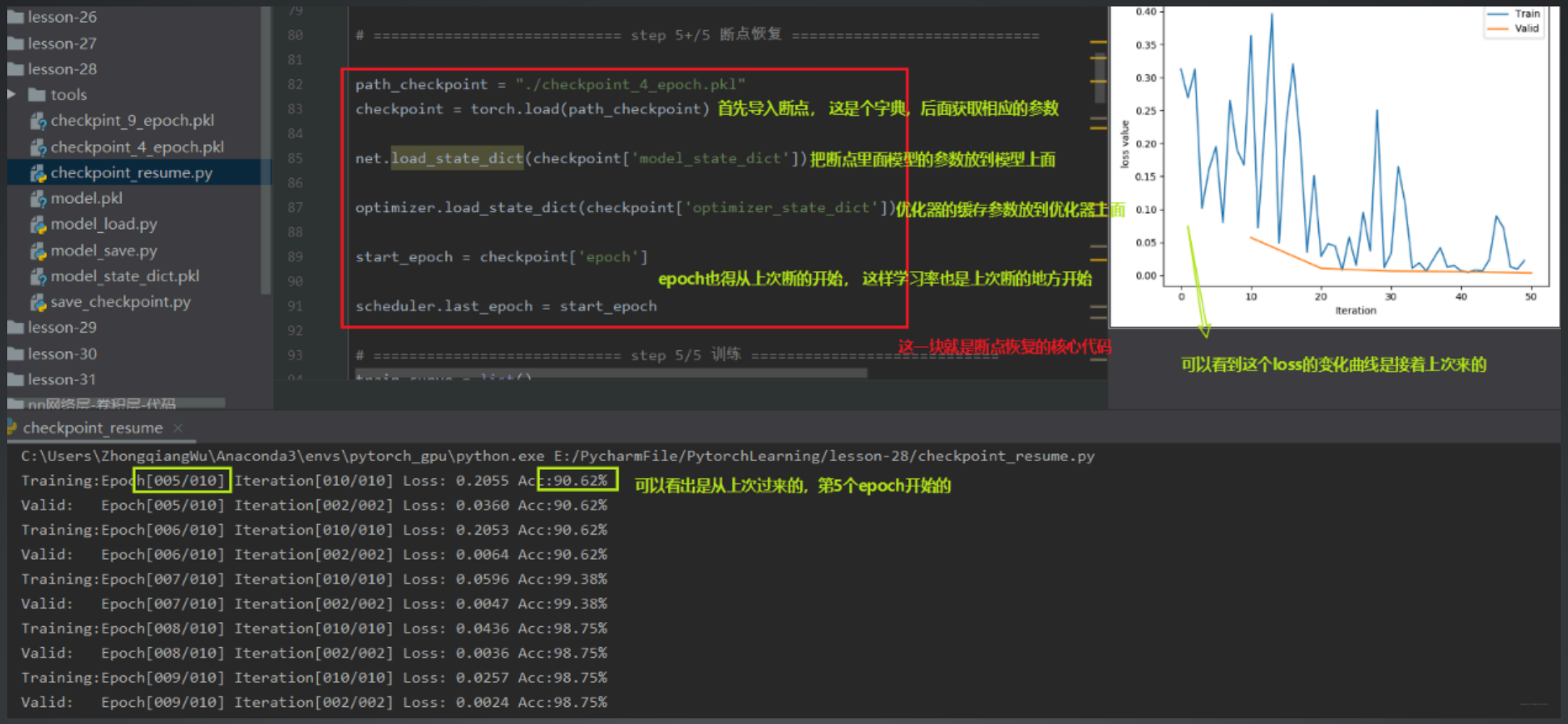
```
checkpoint = {
            "model_state_dict": net.state_dict(),
             "optimizer_state_dict": optimizer.state_dict(),
             "epoch": epoch
}
```

下面通过人民币二分类的实验，模拟一个训练过程中的意外中断和恢复，看看怎么使用这个断点续训练：



行保存，

- 那么我们下面就进行恢复， 从断点处进行训练，也就是上面的第6个epoch开始，我们看看怎么恢复断点训练：

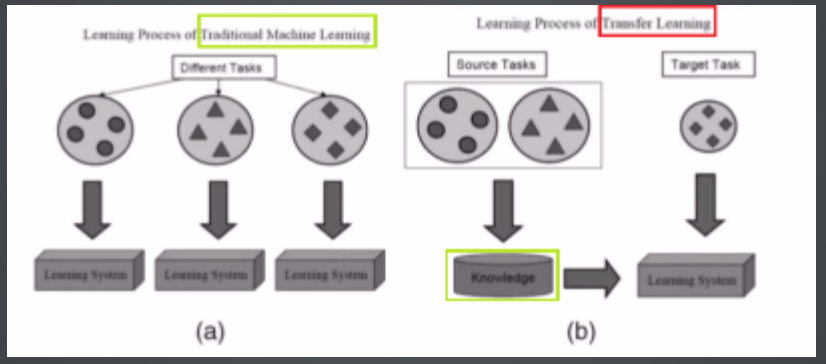


- 所以在模型的训练过程当中， 以一定的间隔去保存我们的模型，保存断点，

- 在断点里面不仅要保存模型的参数，还要保存优化器的参数。这样才可以在意外中断之后恢复训练。

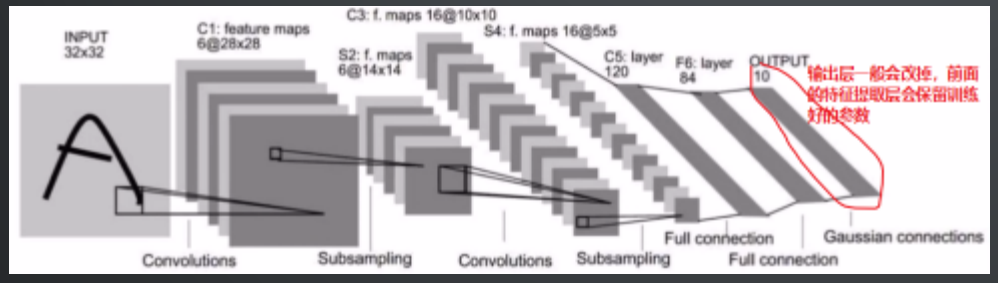
3.9-3. 模型的finetune

在说模型的finetune之前，得先知道一个概念，就是迁移学习。



迁移学习： 机器学习分支， 研究**源域**的知识如何应用到**目标域**，将源任务中学习到的知识运用到目标任务当中，用来提升目标任务里模型的性能。

- 当我们某个任务的数据比较少的时候，没法训练一个好的模型时， 就可以采用迁移学习的思路
- 把**类似任务**训练好的模型给迁移过来，由于这种模型已经在原来的任务上训练的差不多了，迁移到新任务上之后，只需要微调一些参数，往往就能比较好的应用于新的任务
- 可能需要在原来模型的基础上修改输出部分，毕竟任务不同，输出可能不同。
- 但是要注意，只能在类似任务上模型迁移（不要试图将一个NLP的模型迁移到CV里面去）



模型微调的步骤：

1. 获取预训练模型参数（源任务当中学习到的知识）
2. 加载模型（load_state_dict）将学习到的知识放到新的模型
3. 修改输出层， 以适应新的任务

模型微调的训练方法：

- 固定预训练的参数(requires_grad=False; lr=0)
- Features Extractor较小学习率(params_group)

下面通过一个例子，看看如何使用模型的finetune：

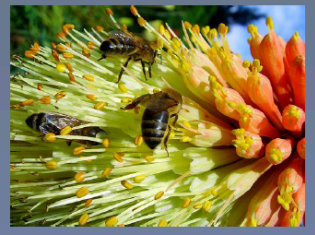
- 使用训练好的ResNet-18进行二分类： 让模型分出蚂蚁和蜜蜂：

注意这张图，里面有作业需要的数据集和被迁移学习的模型

Finetune Resnet-18 用于二分类

蚂蚁蜜蜂二分类数据

训练集：各120～张　　验证集：各70～张





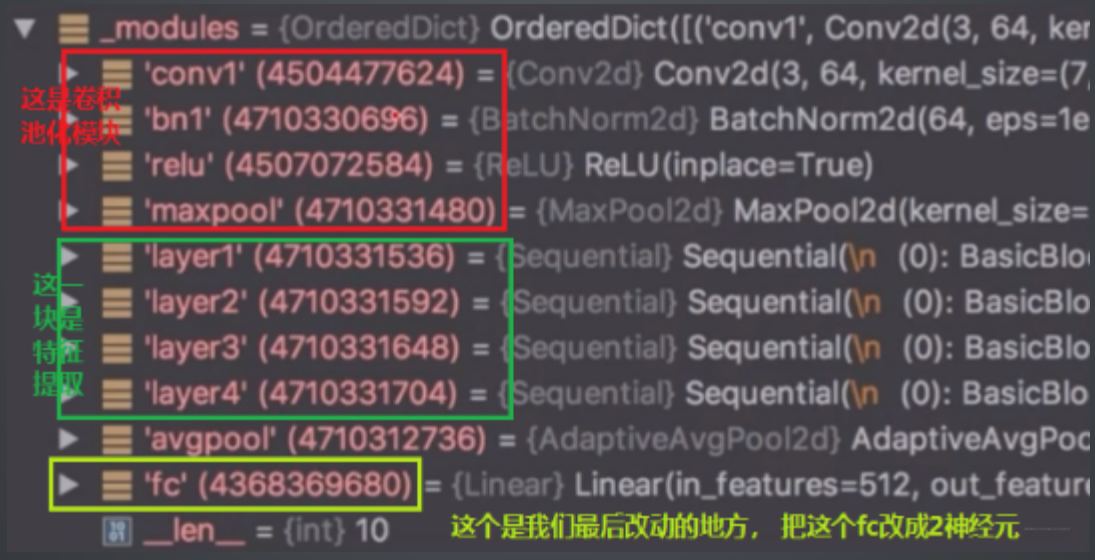
```
_modules = {OrderedDict} OrderedDict([('conv1', Conv2d(3, 64, ker
  'conv1' (4504477624) = {Conv2d} Conv2d(3, 64, kernel_size=(7,
  'bn1' (4710330696) = {BatchNorm2d} BatchNorm2d(64, eps=1e-
  'relu' (4507072584) = {ReLU} ReLU(inplace=True)
  'maxpool' (4710331480) = {MaxPool2d} MaxPool2d(kernel_size=
  'layer1' (4710331536) = {Sequential} Sequential(\n  (0): BasicBloc
  'layer2' (4710331592) = {Sequential} Sequential(\n  (0): BasicBlo
  'layer3' (4710331648) = {Sequential} Sequential(\n  (0): BasicBlo
  'layer4' (4710331704) = {Sequential} Sequential(\n  (0): BasicBloc
  'avgpool' (4710312736) = {AdaptiveAvgPool2d} AdaptiveAvgPoo
  'fc' (4368369680) = {Linear} Linear(in_features=512, out_feature
  __len__ = {int} 10
```

数据：https://download.pytorch.org/tutorial/hymenoptera_data.zip
模型：https://download.pytorch.org/models/resnet18-5c106cde.pth'

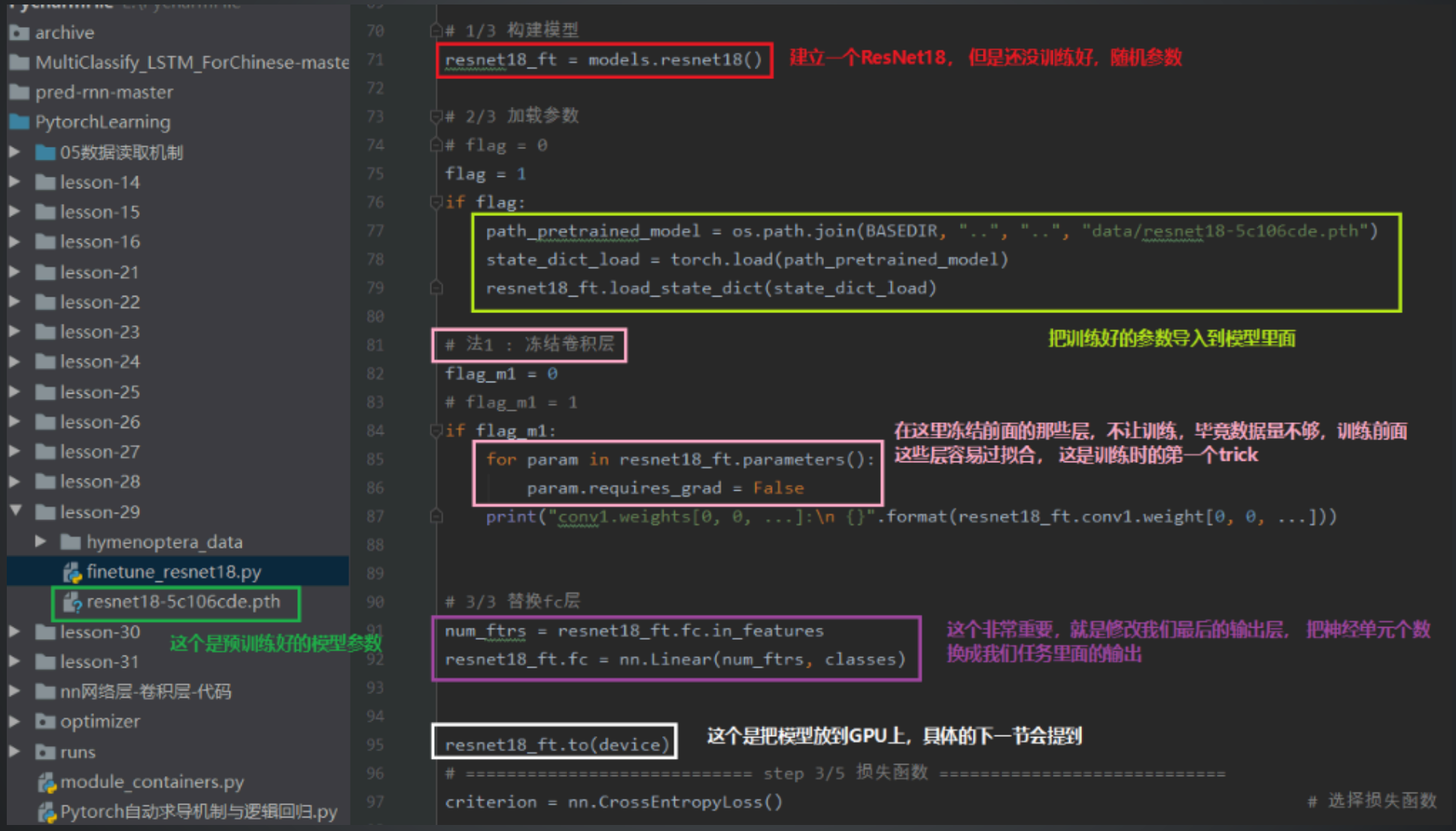
Resnet-18

- 当前有训练集120张， 验证集70张
- 可以看到这里的数据太少了，如果我们新建立模型进行训练预测，估计没法训练。
- 所以看看迁移技术， 我们用训练好的ResNet-18来完成这个任务。

首先看看ResNet-18的结构，看看我们需要在哪里进行改动：



下面看看具体应该怎么使用：



- 冻结卷积层(特征提取层)， 全连接层的参数不能冻结掉，
- 如果都冻结了，那还咋做新的全连接层的训练(增量finetune)
- 所以在遍历网络参数的时候，应该有个判断(if 是卷积层参数)或者是遍历 `resnet18_ft.features.parameters()`。

当然，训练时的trick还有第二个，就是不冻结前面的层，而是修改前面的参数学习率

- 因为优化器里面有参数组的概念，我们可以把网络的前面和后面分成不同的参数组，
- 使用不同的学习率进行训练，当前面的学习率为0的时候，就是和冻结前面的层一样的效果了，但是这种写法比较灵活

```
# ============================ step 4/5 优化器 ============================
# 法2 : conv 小学习率
# flag = 0
flag = 1
if flag:
    fc_params_id = list(map(id, resnet18_ft.fc.parameters()))     # 返回的是parameters的 内存地址
    base_params = filter(lambda p: id(p) not in fc_params_id, resnet18_ft.parameters())
    optimizer = optim.SGD([
        {'params': base_params, 'lr': LR*0},    # 0
        {'params': resnet18_ft.fc.parameters(), 'lr': LR}], momentum=0.9)

else:
    optimizer = optim.SGD(resnet18_ft.parameters(), lr=LR, momentum=0.9)              # 选择优化器

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=lr_decay_step, gamma=0.1)     # 设置学习率下降策略
```

ResNet后面的全连接层

ResNet前面的卷积层

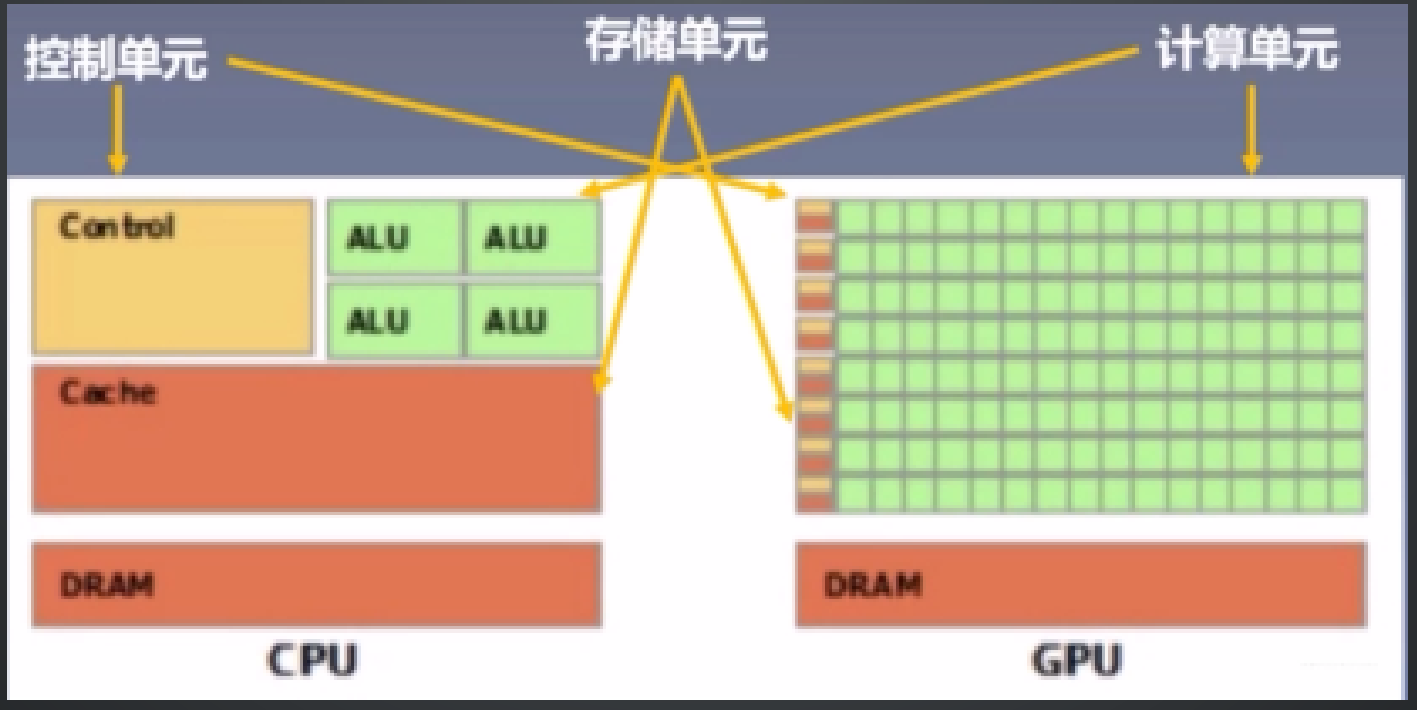
优化器的参数组，可以发现，前面的卷积层学习率比较小，如果想不让训练，可以设置学习率为0，而后面的全连接层的学习率可以大一些

通过模型的迁移，可以发现这个任务就会完成的比较好。

3.9-4. GPU的使用

3.9-4.1 CPU VS GPU

- CPU（Central Processing Unit, 中央处理器）： 主要包括控制器和运算器
- GPU（Graphics Processing Unit, 图形处理器）： 处理统一的， 无依赖的大规模数据运算



3.9-4.2 数据迁移至GPU

首先， 数据主要有两种： Tensor和Module

- GPU -> CPU：data.to(“cpu”)
- CPU -> GPU: data.to(“cuda”)

to函数： 转换数据类型/设备

1. tensor.to(*args, **kwargs)

```
x = torch.ones((3,3))
x = x.to(torch.float64)      # 转换数据类型

x = torch.ones((3,3))
x = x.to("cuda")      # 设备转移
```

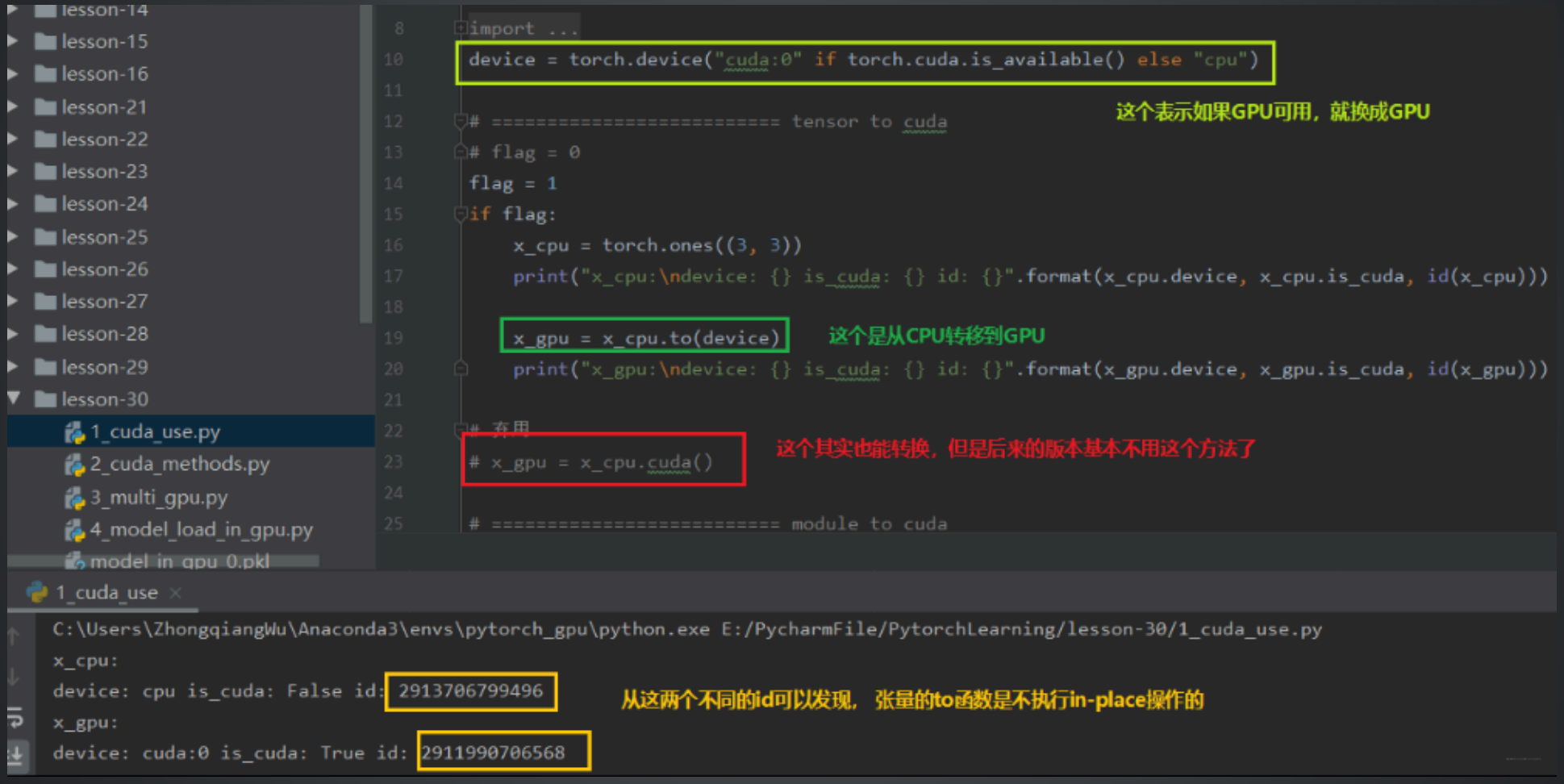
2. module.to(*args, **kwargs)

```
linear = nn.Linear(2,2)
linear.to(torch.double)   # 这样模型里面的可学习参

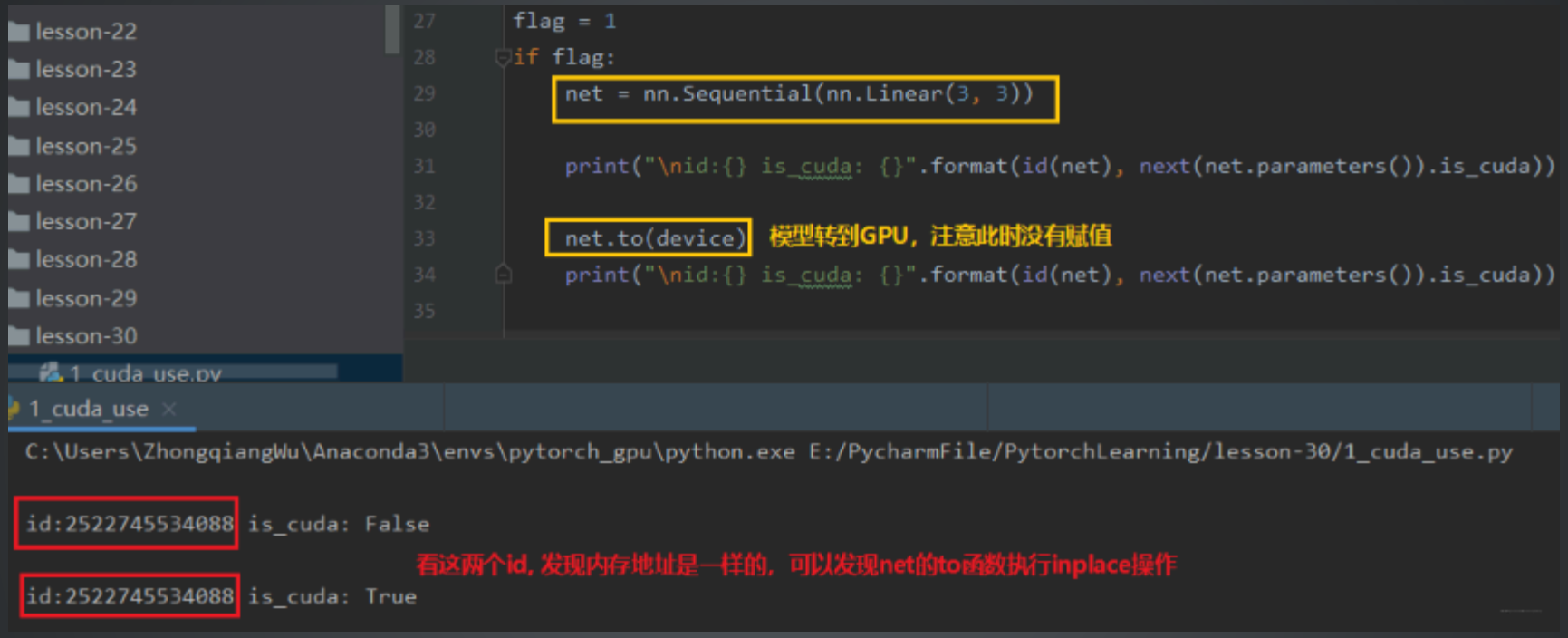
gpu1 = torch.device("cuda")
linear.to(gpu1)      # 把模型从CPU迁移到GPU
```

上面两个方法的区别： 张量**不执行inplace**， 所以上面看到需要等号重新赋值，而模型**执行inplace**， 所以不用等号重新赋值。

下面从代码中学习上面的两个方法：



下面看一下Module的to函数：



- 如果模型在GPU上， 那么数据也必须在GPU上才能正常运行。
- 也就是说**数据和模型必须在相同的设备上**。

提问：在Pytorch中，如何将张量从CPU迁移到GPU?

- A) `tensor.to("cpu")`
- B) `tensor.to("cuda")`
- C) `torch.save(tensor)`
- D) `torch.load(tensor)`

参考答案：B

选项解析：

- A) `tensor.to("cpu")`用于将张量迁移到CPU。
- B) `tensor.to("cuda")`用于将张量迁移到GPU。
- C) `torch.save(tensor)`用于保存张量，不涉及设备迁移。
- D) `torch.load(tensor)`用于加载张量，不涉及设备迁移。

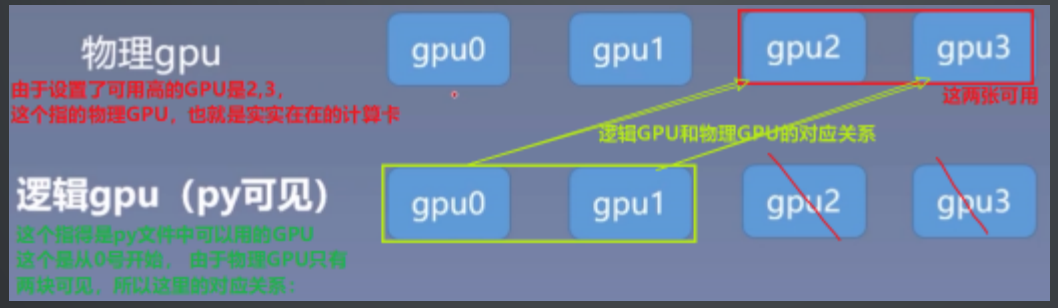
torch.cuda常用的方法：

1. `torch.cuda.device_count()`：计算当前**可见可用**的GPU数
2. `torch.cuda.get_device_name()`：获取GPU名称
3. `torch.cuda.manual_seed()`：为当前GPU设置随机种子
4. `torch.cuda.manual_seed_all()`：为所有可见可用GPU设置随机种子
5. `torch.cuda.set_device()`：设置主GPU（默认GPU）为哪一个物理GPU（不推荐）

推荐的选择GPU方式：设置系统的环境变量

`os.environ.setdefault("CUDA_VISIBLE_DEVICES", "2,3")` 通过这个方法合理的分配GPU，使得多个人使用的时候不冲突。

- 但是要注意， 这里的2,3指的是物理GPU的2,3。但是在逻辑GPU上， 这里表示的0,1。 这里看一个对应关系吧：



3.9-5. 总结

这篇文章到这里也就结束了，也就意味着Pytorch的基础知识，基本概念也都整理完毕，

快速回顾一下这次学习的知识

- 首先学习了模型的保存与加载问题，介绍了两种模型保存与加载的方法，
- 然后学习了模型的微调技术，这个在迁移学习中用处非常大，
- 还介绍了迁移学习中常用的两个trick。
- 然后学习了如何使用GPU加速训练和GPU并行训练方式

到这里为止，关于Pytorch的基本知识结束。下面也对这十篇文章进行一个梳理和总结，

- 这十篇文章的逻辑其实也非常简单，就是围绕着机器学习模型训练的五大步骤进行展开的：

1. 先学习了一下Pytorch的基本知识，知道了什么是张量
2. 然后学习了自动求导系统，计算图机制， 对Pytorch有了一个基本的了解之后
3. 我们就开始学习Pytorch的数据读取机制，在里面知道了DataLoader和Dataset，还学习了图像预处理的模块transform
4. 接着学习模型模块，知道了如何去搭建一个模型，一个模型是怎么去进行初始化的
5. 还学习了容器，常用网络层的使用
6. 再往后就是网络层的权重初始化方法和8种损失函数

7. 有了损失函数之后，接着就开始学习各种优化器帮助我们更新参数, 还有学习率调整的各种策略。
8. 有了数据，模型，损失，优化器，就可以迭代训练模型了， 所以在迭代训练过程中学习了Tensorboard这个非常强大的可视化工具，可以帮助我们更好的监控模型训练的效果
9. 然后学习了正则化和标准化技术， 正则化可以帮助缓解模型的过拟合，这里面学习了L1,L2和Dropout的原理和使用，而标准化可以更好的解决数据尺度不平衡的问题， 这里面有BN， LN， IN， GN四种标准化方法，并对比了它们的不同及应用场景
10. 最后收尾，学习了模型的保存加载，模型微调，如何使用GPU

下面放一张图，这个代表着深度学习的相关领域

